République tunisienne
Ministère de l'Enseignement supérieur et de la Recherche scientifique
TEK-UP University
École privée supérieure de technologie et d'ingénierie

Conception et Implémentation d'un Système Intelligent de Gestion d'Atelier Automobile

Présenté en vue de l'obtention de la

Cycle d'Ingénieur en Informatique

Parcours : Génie Logiciel

Réalisé par

Ihebeddine Saafi | Arbi Hazbri | Louay Lewati

MecaManage

Thème du projet

Année Universitaire : 2025–2026

Table des matières

Introduction Générale **3**

1 Étude Préalable et Spécification des Besoins **5**

1.1 Analyse de l'Existant 5

1.1.1 Solutions Actuelles sur le Marché 5

1.1.2 Limites Identifiées 5

1.2 Identification des Acteurs 6

1.3 Spécification des Besoins 6

1.3.1 Besoins Fonctionnels 6

1.3.2 Besoins Non Fonctionnels 7

1.4 Diagramme de Cas d'Utilisation 7

1.4.1 Description des Cas d'Utilisation Principaux 9

1.5 Diagramme de Classes 10

1.5.1 Description des Entités Principales 12

1.6 Architecture du Système 12

1.6.1 Vue d'Ensemble — Architecture Microservices 12

1.6.2 Composants de l'Architecture 13

1.6.3 Stack Technique Récapitulatif 14

Conclusion du Chapitre 1 15

2 Gestion de Projet Agile — Scrum **16**

2.1 Présentation de la Méthodologie Scrum 16

2.2 Rôles Scrum 17

2.3 Cérémonies Scrum 17

2.4 Backlog Produit 17

2.4.1 Conventions d'Estimation 18

2.4.2 Backlog Produit Complet 18

2.5 Planification des Sprints 23

2.6 Diagramme de Gantt — Planification des Sprints 23

2.7 Vélocité et Suivi d'Avancement 23

Conclusion du Chapitre 2 24

Table des figures

1.1 Diagramme de cas d'utilisation global — MecaManage 8
1.2 Diagramme de classes — Domaine MecaManage 11
1.3 Architecture microservices — MecaManage 13

2.1 Diagramme de Gantt — Planification des sprints MecaManage 23

Introduction Générale

Contexte et Motivation

L'industrie automobile connaît une transformation profonde portée par la numérisation des processus métiers, la montée en puissance de l'intelligence artificielle et l'exigence croissante de qualité de service de la part des clients. Les garages et ateliers mécaniques font face à des défis opérationnels majeurs : gestion inefficace des rendez-vous, suivi manuel des réparations, absence de traçabilité des stocks et difficulté à exploiter les données historiques.

La majorité des solutions existantes sur le marché restent soit trop génériques, soit trop rudimentaires pour répondre aux besoins d'entreprises multi-établissements. C'est dans cette perspective que s'inscrit le projet **MecaManage** : une plateforme SaaS (*Software as a Service*) de gestion d'ateliers mécaniques conçue pour les entreprises exploitant plusieurs garages.

Problématique

Les entreprises de garages multi-établissements sont confrontées à plusieurs problèmes récurrents :

- **Fragmentation des données** : chaque garage gère son propre stock et ses plannings sans vision consolidée.
- **Absence d'intelligence diagnostique** : le diagnostic repose entièrement sur l'expertise humaine sans capitalisation.
- **Manque de traçabilité** : le suivi des réparations n'est pas accessible en temps réel aux clients.
- **Processus manuels coûteux** : devis, notifications et relances sont traités manuellement.
- **Absence de reporting consolidé** : pas de tableau de bord agrégé multi-garage.

Objectifs du Projet

1. Fournir une **plateforme SaaS multi-tenant** pour la gestion centralisée de plusieurs garages.
2. Intégrer un **module de diagnostic assisté par IA** pour l'analyse des symptômes.
3. Automatiser les **workflows métiers** via n8n.
4. Offrir un **suivi en temps réel** des interventions.
5. Mettre en place une **infrastructure DevOps industrialisée** garantissant disponibilité, sécurité et scalabilité.

Structure du Rapport

Chapitre 1 Étude préalable et spécification des besoins : analyse de l'existant, acteurs, UML, architecture.

Chapitre 2 Conception détaillée : microservices, modélisation des données, interfaces.

Chapitre 3 Implémentation : ASP.NET Core 8, Angular 17, Python FastAPI.

Chapitre 4 Infrastructure et DevOps : Docker, CI/CD, Ansible, monitoring.

Conclusion Bilan, limites et perspectives.

Chapitre 1

Étude Préalable et Spécification des Besoins

Introduction

Ce premier chapitre pose les fondations du projet MecaManage. Après une analyse de l'existant et une identification des besoins, nous présentons les acteurs, les cas d'utilisation, le diagramme de classes et l'architecture technique retenue.

1.1 Analyse de l'Existant

1.1.1 Solutions Actuelles sur le Marché

Table 1.1 – Comparaison des solutions existantes

Critère	**Solutions génériques**	**ERP automobiles**	**MecaManage**
Multi-établissements	Limité	Partiel	✓ Complet
Diagnostic IA	✗	✗	✓ Intégré
SaaS natif	Rare	Non	✓ Natif
Automatisation workflows	✗	Partiel	✓ n8n
Suivi temps réel	✗	✗	✓ Inclus
DevOps / Open API	✗	✗	✓ Docker + CI/CD

1.1.2 Limites Identifiées

— Absence de capitalisation sur les données historiques de réparation pour alimenter un module de diagnostic automatisé.

— Architecture monolithique incompatible avec des déploiements SaaS multi-tenant.
— Pas d'API ouverte permettant l'intégration avec des outils tiers (n8n, Prometheus, etc.).
— Interface client rudimentaire, sans portail de suivi en temps réel.

1.2 Identification des Acteurs

Table 1.2 – Acteurs du système MecaManage

Acteur	Rôle	Portée
SuperAdmin	Gestion globale de la plateforme SaaS, tenants	Plateforme
Admin Entreprise	Gestion garages, reporting, configuration	Entreprise
Chef d'Atelier	Dossiers, affectation mécaniciens, devis	Garage
Mécanicien	Exécution des interventions assignées	Garage
Client	Demandes, suivi réparations en temps réel	Portail client

1.3 Spécification des Besoins

1.3.1 Besoins Fonctionnels

Module Gestion Clients et Véhicules

— Création et authentification des comptes clients (JWT + Refresh Tokens).
— Enregistrement des véhicules avec historique complet.
— Soumission de demandes via formulaire intelligent (photos, kilométrage, description libre des symptômes).

Module Diagnostic IA

— Analyse NLP des symptômes décrits en langage naturel.
— Production d'un diagnostic probable avec score de confiance.
— Recommandation du type d'atelier requis (mécanique, électrique, hybride).

Module Gestion des Interventions

— Affectation des dossiers par l'admin aux chefs d'atelier.
— Acceptation / refus avec notification automatique client.

— Suivi du statut en temps réel : *En attente* → *Accepté* → *En cours* → *En attente pièces* → *Terminé*.

— Génération de devis dynamiques en cas de remplacement de pièces.

Module Stock

— Gestion du stock par établissement avec traçabilité.

— Réservation automatique des pièces lors de l'acceptation d'un devis.

— Alertes de stock minimum via n8n.

Module Facturation et Reporting

— Génération automatique des factures en fin d'intervention.

— Tableau de bord Admin : CA global, performance par garage, taux d'acceptation devis, rendement mécaniciens.

1.3.2 Besoins Non Fonctionnels

TABLE 1.3 – Besoins non fonctionnels

Catégorie	Exigence
Performance	Temps de réponse API < 200 ms (p95), 500 utilisateurs simultanés
Disponibilité	SLA 99,9 % — déploiement zéro-downtime via Ansible
Sécurité	HTTPS TLS 1.3, JWT, isolation réseau Docker, Rate Limiting
Scalabilité	Architecture microservices horizontalement scalable
RGPD	Droit à l'effacement, export données, consentements
Maintenabilité	Clean Architecture, couverture tests > 80 %, docs OpenAPI

1.4 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation global (Figure 1.1) représente les interactions entre les cinq acteurs et les principaux cas d'utilisation du système.

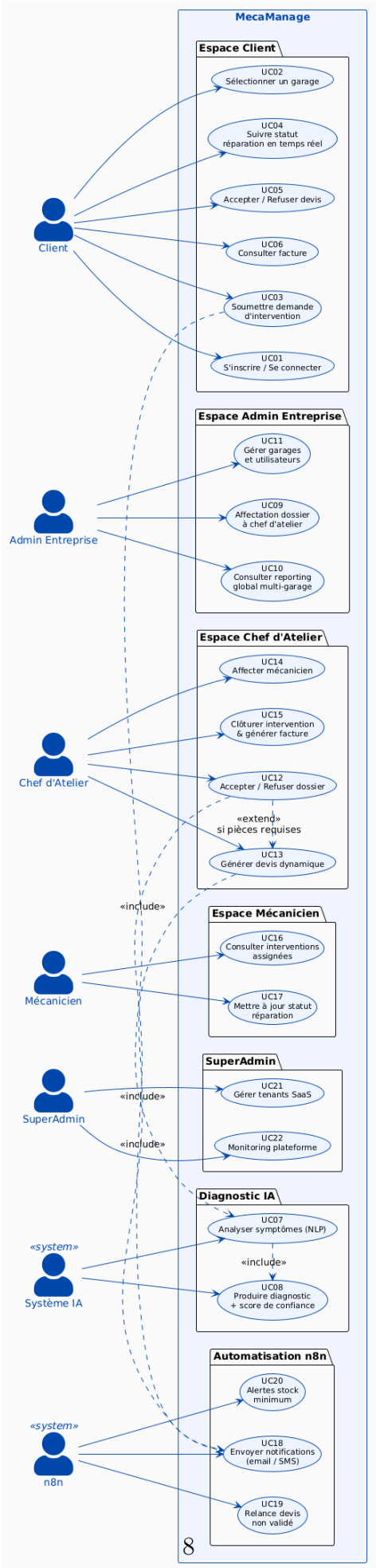


FIGURE 1.1 – Diagramme de cas d'utilisation global — MecaManage

1.4.1 Description des Cas d'Utilisation Principaux

UC01 — S'inscrire et Sélectionner un Garage

Table 1.4 – Fiche UC01 — S'inscrire et sélectionner un garage

Champ	**Description**
Acteur principal	Client
Précondition	Aucune
Scénario nominal	1. Le client accède au portail et crée son compte. 2. Le système détecte sa localisation géographique. 3. Le client visualise les garages disponibles. 4. Il sélectionne un garage selon la distance ou la disponibilité.
Postcondition	Compte créé, garage sélectionné, session ouverte.
Exceptions	Aucun garage disponible → créneau ultérieur proposé.

UC02 — Soumettre une Demande d'Intervention

Table 1.5 – Fiche UC02 — Soumettre une demande d'intervention

Champ	**Description**
Acteur principal	Client
Précondition	Client authentifié, garage sélectionné.
Scénario nominal	1. Le client remplit le formulaire (véhicule, km, photos, symptômes). 2. Il soumet la demande. 3. Le microservice IA analyse les symptômes (`«include»`). 4. Un diagnostic probable est attaché au dossier. 5. L'administrateur est notifié automatiquement.
Postcondition	Dossier créé avec diagnostic IA, admin notifié.
Extensions	`«include»` Analyser symptômes par IA.

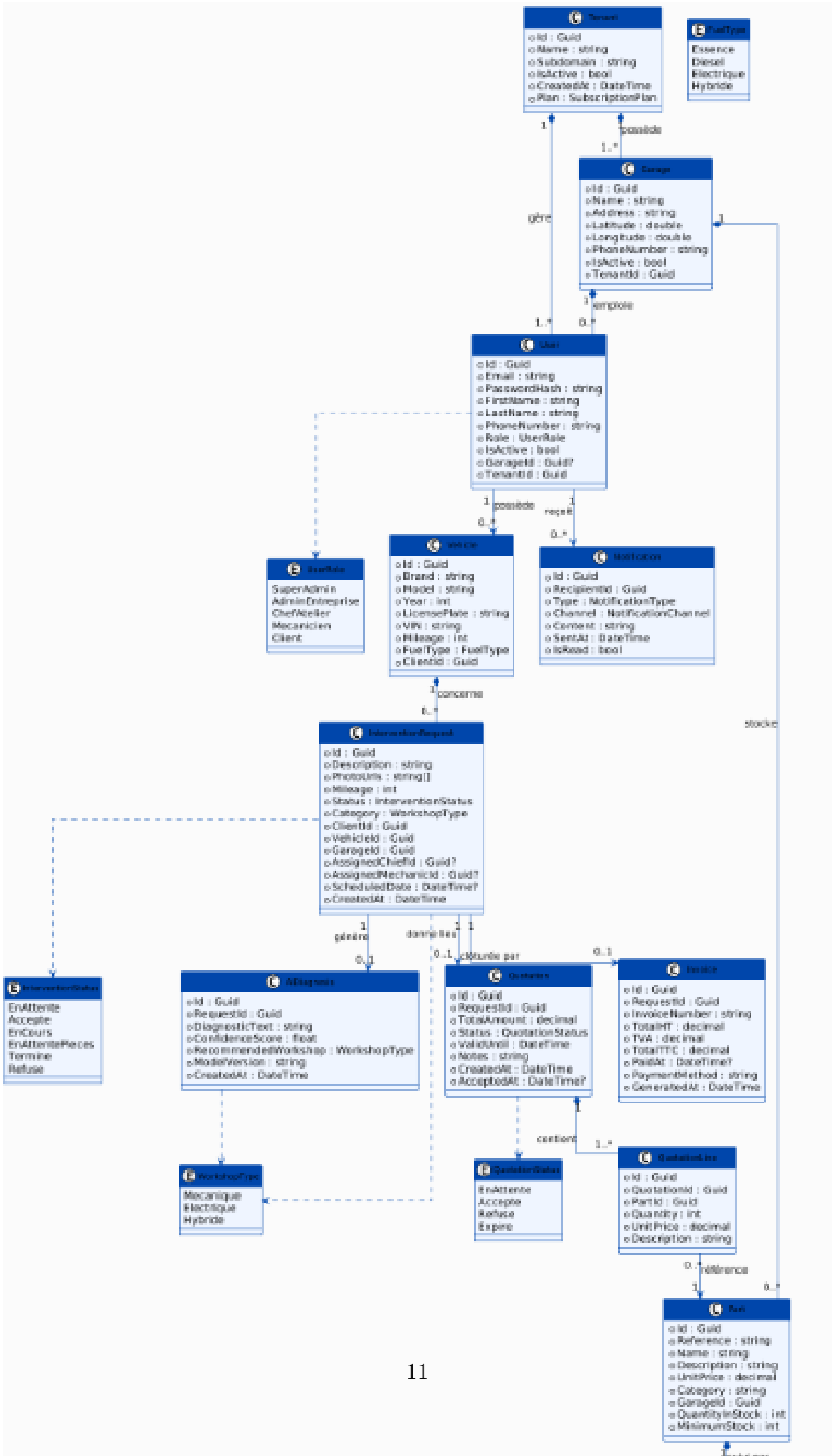
UC03 — Gérer les Dossiers (Chef d'Atelier)

Table 1.6 – Fiche UC03 — Gérer les dossiers

Champ	**Description**
Acteur principal	Chef d'Atelier
Précondition	Dossier affecté par l'admin.
Scénario nominal	1. Le chef d'atelier consulte le dossier et le diagnostic IA. 2. Il accepte ou refuse le dossier. 3. Si accepté : date fixée, client notifié (email + SMS). 4. Si pièces nécessaires : devis généré et envoyé au client. 5. Fin d'intervention : facture générée automatiquement.
Postcondition	Client notifié, workflow mis à jour.
Extensions	`«extend»` Générer devis si pièces nécessaires.

1.5 Diagramme de Classes

Le diagramme de classes (Figure 1.2) modélise les entités principales du domaine métier et leurs relations, suivant les principes de la Clean Architecture.


Tenant
Id : Guid
Name : string
Subdomain : string
IsActive : bool
CreatedAt : DateTime
Plan : SubscriptionPlan
FuelType
Essence
Diesel
Electrique
Hybride
possède
gère
Garage
Id : Guid
Name : string
Address : string
Latitude : double
Longitude : double
PhoneNumber : string
IsActive : bool
TenantId : Guid
emploie
User
Id : Guid
Email : string
PasswordHash : string
FirstName : string
LastName : string
PhoneNumber : string
Role : UserRole
IsActive : bool
GarageId : Guid?
TenantId : Guid
possède
reçoit
UserRole
SuperAdmin
AdminEntreprise
ChefAtelier
Mecanicien
Client
Vehicle
Id : Guid
Brand : string
Model : string
Year : int
LicensePlate : string
VIN : string
Mileage : int
FuelType : FuelType
ClientId : Guid
Notification
Id : Guid
RecipientId : Guid
Type : NotificationType
Channel : NotificationChannel
Content : string
SentAt : DateTime
IsRead : bool
concerne
stocke
InterventionRequest
Id : Guid
Description : string
PhotoUrls : string[]
Mileage : int
Status : InterventionStatus
Category : WorkshopType
ClientId : Guid
VehicleId : Guid
GarageId : Guid
AssignedChiefId : Guid?
AssignedMechanicId : Guid?
ScheduledDate : DateTime?
CreatedAt : DateTime
InterventionStatus
EnAttente
Accepte
EnCours
EnAttentePieces
Termine
Refuse
génère
AIDiagnosis
Id : Guid
RequestId : Guid
DiagnosticText : string
ConfidenceScore : float
RecommendedWorkshop : WorkshopType
ModelVersion : string
CreatedAt : DateTime
WorkshopType
Mecanique
Electrique
Hybride
Quotation
Id : Guid
RequestId : Guid
TotalAmount : decimal
Status : QuotationStatus
ValidUntil : DateTime
Notes : string
CreatedAt : DateTime
AcceptedAt : DateTime?
Invoice
Id : Guid
RequestId : Guid
InvoiceNumber : string
TotalHT : decimal
TVA : decimal
TotalTTC : decimal
PaidAt : DateTime?
PaymentMethod : string
GeneratedAt : DateTime
contient
QuotationStatus
EnAttente
Accepte
Refuse
Expire
QuotationLine
Id : Guid
QuotationId : Guid
PartId : Guid
Quantity : int
UnitPrice : decimal
Description : string
référence
Part
Id : Guid
Reference : string
Name : string
Description : string
UnitPrice : decimal
Category : string
GarageId : Guid
QuantityInStock : int
MinimumStock : int

1.5.1 Description des Entités Principales

Tenant Entreprise cliente de la plateforme SaaS. Chaque tenant possède un ou plusieurs garages.

Garage Établissement physique avec son propre stock, ses chefs d'atelier et ses mécaniciens.

User Entité de base : SuperAdmin, AdminEntreprise, ChefAtelier, Mécanicien, Client.

Vehicle Véhicule associé à un client avec historique complet.

InterventionRequest Demande d'intervention incluant le diagnostic IA et le statut courant.

AIDiagnosis Résultat IA : diagnostic textuel, score de confiance, type d'atelier recommandé.

Quotation Devis généré par le chef d'atelier pour une intervention nécessitant des pièces.

Invoice Facture finale générée automatiquement à la clôture.

Part Pièce de rechange dans le stock d'un garage.

StockMovement Traçabilité des mouvements de stock (entrée / sortie / réservation).

1.6 Architecture du Système

1.6.1 Vue d'Ensemble — Architecture Microservices

MecaManage adopte une architecture microservices conteneurisée (Figure 1.3) garantissant la scalabilité horizontale, l'isolation des composants et la résilience.

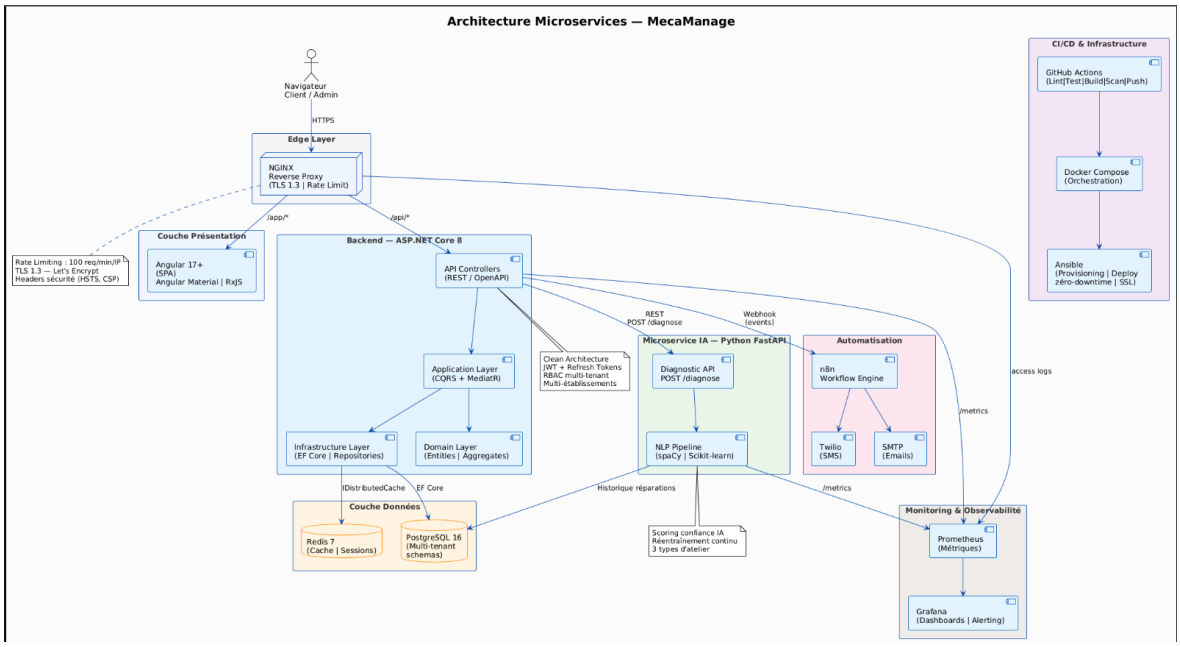


FIGURE 1.3 – Architecture microservices — MecaManage

1.6.2 Composants de l'Architecture

Couche Présentation

— **Angular 17+** avec Angular Material et RxJS — interface client et portail d'administration (SPA).

— Déployé via un conteneur NGINX servant les assets statiques compilés.

API Gateway — NGINX Reverse Proxy

— Routage des requêtes vers les microservices appropriés.

— Terminaison TLS 1.3, Rate Limiting, compression gzip.

Backend Principal — ASP.NET Core 8

— **Clean Architecture** : Domain / Application / Infrastructure / API.

— Pattern **CQRS** + **MediatR** pour la séparation des commandes et des requêtes.

— **Entity Framework Core** avec PostgreSQL 16.

— Authentification **JWT** + **Refresh Tokens**, RBAC multi-rôles et multi-tenant.

— Cache **Redis 7** pour sessions et données fréquemment accédées.

Microservice IA — Python FastAPI

— API REST pour l'analyse des symptômes en langage naturel.

— Pipeline NLP : prétraitement → vectorisation → classification (Scikit-learn + spaCy).

— Réentraînement continu à partir des données validées par les chefs d'atelier.

Automatisation — n8n

— Rappels RDV, notifications acceptation/refus, relances devis, alertes stock, rapports hebdomadaires.

— Intégration SMTP, Twilio (SMS), webhooks API principale.

Infrastructure DevOps

— **Docker + Docker Compose** pour l'orchestration.

— **GitHub Actions** : lint → tests → build → scan sécurité → push registry → déploiement.

— **Ansible** : provisioning, déploiement zéro-downtime, SSL Let's Encrypt.

— **Prometheus + Grafana** : monitoring, alerting, dashboards.

1.6.3 Stack Technique Récapitulatif

Table 1.7 – Stack technique MecaManage

Couche	Technologie	Rôle
Frontend	Angular 17 + Material	Interface utilisateur SPA
Backend	ASP.NET Core 8	API REST, logique métier, CQRS
Base de données	PostgreSQL 16	Persistance relationnelle
Cache	Redis 7	Sessions, cache applicatif
IA	Python FastAPI + Scikit-learn	Diagnostic automatisé NLP
Automatisation	n8n	Workflows métiers automatisés
Reverse Proxy	NGINX	Routage, TLS, rate limiting
Conteneurisation	Docker + Compose	Isolation et portabilité
CI/CD	GitHub Actions	Intégration et déploiement continus
IaC	Ansible	Provisioning et déploiement
Monitoring	Prometheus + Grafana	Observabilité système

Conclusion du Chapitre 1

Ce chapitre a établi les fondations analytiques et conceptuelles du projet MecaManage. L'analyse de l'existant a mis en évidence les manques structurels des solutions actuelles, justifiant une approche sur mesure. La spécification des besoins a guidé le choix d'une architecture microservices basée sur ASP.NET Core 8, Angular 17 et Python FastAPI. Les diagrammes UML formalisent les contrats du système, tandis que l'architecture retenue garantit scalabilité, résilience et maintenabilité. Le chapitre suivant se consacrera à la conception détaillée.

Chapitre 2

Gestion de Projet Agile — Scrum

Introduction

Dans le cadre du développement de MecaManage, nous avons adopté la méthodologie agile **Scrum** afin de structurer le travail en itérations courtes, de livrer de la valeur de manière incrémentale et de s'adapter aux évolutions des besoins. Ce chapitre présente les rôles Scrum, le backlog produit, la planification des sprints et le suivi de l'avancement.

2.1 Présentation de la Méthodologie Scrum

Scrum est un cadre agile basé sur des cycles itératifs appelés **sprints**, d'une durée fixe de deux à quatre semaines. Il repose sur trois piliers fondamentaux :

- **Transparence** : toutes les informations relatives à l'avancement sont visibles par l'équipe et les parties prenantes.
- **Inspection** : l'équipe inspecte régulièrement l'avancement lors des cérémonies Scrum.
- **Adaptation** : le plan est ajusté en fonction des retours obtenus à chaque sprint.

2.2 Rôles Scrum

Table 2.1 – Rôles Scrum dans le projet MecaManage

Rôle Scrum	Membre	Responsabilités
Product Owner	Ihebeddine Saafi	Définit et priorise le backlog produit, valide les livraisons, représente les besoins métier.
Scrum Master	Arbi Hazbri	Facilite les cérémonies Scrum, lève les obstacles, veille au respect du cadre agile.
Développeur	Louay Lewati	Conception, développement, tests et intégration des fonctionnalités du système.
Développeur	Toute l'équipe	Participation aux sprints, revues de code, documentation technique.

2.3 Cérémonies Scrum

Table 2.2 – Cérémonies Scrum adoptées

Cérémonie	Objectif	Fréquence
Sprint Planning	Sélectionner les user stories à réaliser durant le sprint et estimer l'effort.	Début de sprint
Daily Scrum	Synchronisation quotidienne : avancements, blocages, plan du jour.	Quotidien (15 min)
Sprint Review	Démonstration des fonctionnalités livrées aux parties prenantes.	Fin de sprint
Sprint Retrospective	Analyse du processus : ce qui a bien fonctionné, axes d'amélioration.	Fin de sprint
Backlog Refinement	Affiner, détailler et réestimer les user stories du backlog.	Mi-sprint

2.4 Backlog Produit

Le backlog produit regroupe l'ensemble des user stories du projet MecaManage, priorisées selon la méthode **MoSCoW** (Must have, Should have, Could have, Won't have) et estimées en **Story Points** via la séquence de Fibonacci.

2.4.1 Conventions d'Estimation

Table 2.3 – Correspondance Story Points — Complexité

Story Points	Complexité
1	Trivial — configuration, page statique
2	Simple — CRUD basique
3	Modéré — logique métier simple
5	Complexe — intégration multi-composants
8	Très complexe — module complet avec tests
13	Critique — architecture, sécurité, IA

2.4.2 Backlog Produit Complet

Table 2.4: Backlog produit MecaManage

ID	User Story	Critères d'acceptation	SP	Priorité	Sprint
EPIC 1 — Authentification et Gestion Multi-tenant					
US01	En tant que client, je veux créer un compte pour accéder à la plateforme.	Formulaire valide, email unique, mot de passe haché, confirmation email.	3	Must	S1
US02	En tant qu'utilisateur, je veux me connecter avec email et mot de passe pour accéder à mon espace.	JWT généré, refresh token stocké, redirection selon le rôle.	3	Must	S1
US03	En tant qu'utilisateur, je veux que ma session soit renouvelée automatiquement sans me reconnecter.	Refresh token valide, nouveau JWT retourné, expiration gérée.	3	Must	S1
US04	En tant que SuperAdmin, je veux créer un nouveau tenant (entreprise) sur la plateforme.	Tenant créé avec sous-domaine unique, admin assigné, plan sélectionné.	5	Must	S1

Suite page suivante...

ID	User Story	Critères d'acceptation	SP	Priorité	Sprint
US05	En tant que SuperAdmin, je veux activer ou désactiver un tenant pour contrôler l'accès.	Tenant désactivé bloque toutes les connexions associées.	2	Must	S1
US06	En tant qu'Admin Entreprise, je veux créer et gérer mes garages sur la plateforme.	Garage créé avec adresse, coordonnées GPS, statut actif.	5	Must	S1
US07	En tant qu'Admin Entreprise, je veux ajouter des utilisateurs (chefs d'atelier, mécaniciens) à mes garages.	Utilisateur créé avec rôle correct, associé au bon garage, invitation email envoyée.	5	Must	S1
EPIC 2 — Gestion Clients et Véhicules					
US08	En tant que client, je veux ajouter mon véhicule avec ses informations complètes.	Marque, modèle, année, immatriculation, VIN, carburant enregistrés.	3	Must	S2
US09	En tant que client, je veux consulter l'historique complet des interventions de mon véhicule.	Liste triée par date, détail de chaque intervention accessible.	3	Must	S2
US10	En tant que client, je veux sélectionner le garage le plus proche ou le plus disponible.	Carte affichée, distance calculée, disponibilité indiquée.	5	Must	S2
US11	En tant qu'Admin, je veux consulter la liste de tous les clients de mon entreprise.	Liste paginable, filtrable par garage, exportable en CSV.	3	Should	S2
EPIC 3 — Demandes d'Intervention et Diagnostic IA					
US12	En tant que client, je veux soumettre une demande d'intervention avec photos et description des symptômes.	Formulaire multi-étapes, upload photos, validation, dossier créé.	5	Must	S3

Suite page suivante...

ID	User Story	Critères d'acceptation	SP	Priorité	Sprint
US13	En tant que système, je veux analyser automatiquement les symptômes soumis par le client via l'IA.	Microservice FastAPI appelé, réponse en moins de 3 sec, diagnostic attaché au dossier.	13	Must	S3
US14	En tant qu'Admin, je veux voir le diagnostic IA associé à chaque dossier avant affectation.	Diagnostic visible, score de confiance affiché, type atelier recommandé.	3	Must	S3
US15	En tant qu'Admin, je veux affecter un dossier à un chef d'atelier disponible.	Liste chefs disponibles affichée, affectation enregistrée, notification envoyée.	5	Must	S3
US16	En tant que client, je veux être notifié par email et SMS dès qu'un chef accepte mon dossier.	Email et SMS reçus dans les 2 min, contiennent nom du chef et date prévue.	5	Must	S3
EPIC 4 — Workflow de Réparation					
US17	En tant que chef d'atelier, je veux accepter ou refuser un dossier qui m'est affecté.	Statut mis à jour, client notifié, refus renvoyé à l'admin.	3	Must	S4
US18	En tant que chef d'atelier, je veux affecter un mécanicien à une intervention.	Mécanicien notifié, intervention visible dans son tableau de bord.	3	Must	S4
US19	En tant que mécanicien, je veux mettre à jour le statut d'une intervention en temps réel.	Statut mis à jour (En cours, En attente pièces, Terminé), client notifié automatiquement.	5	Must	S4
US20	En tant que client, je veux suivre le statut de mon véhicule en temps réel depuis mon portail.	Statut affiché en temps réel, historique des changements visible.	5	Must	S4
EPIC 5 — Devis et Gestion de Stock					

Suite page suivante...

ID	User Story	Critères d'acceptation	SP	Priorité	Sprint
US21	En tant que chef d'atelier, je veux générer un devis dynamique si des pièces sont nécessaires.	Devis créé avec lignes pièces, montant calculé, envoyé au client.	8	Must	S5
US22	En tant que client, je veux accepter ou refuser un devis depuis mon portail.	Réponse enregistrée, si accepté pièces réservées, si refusé clôture partielle.	5	Must	S5
US23	En tant que chef d'atelier, je veux consulter le stock du garage et rechercher une pièce.	Liste paginable, filtre par référence et catégorie, stock disponible affiché.	3	Must	S5
US24	En tant qu'Admin, je veux être alerté automatiquement quand le stock d'une pièce passe sous le seuil minimum.	Alerte email envoyée via n8n, pièce concernée identifiée.	5	Must	S5
US25	En tant qu'Admin, je veux enregistrer les entrées de stock avec traçabilité complète.	Mouvement enregistré avec date, quantité, utilisateur, motif.	3	Should	S5
EPIC 6 — Facturation et Reporting					
US26	En tant que système, je veux générer automatiquement une facture à la clôture d'une intervention.	Facture PDF générée, numérotée, envoyée par email, accessible dans le portail client.	8	Must	S6
US27	En tant qu'Admin Entreprise, je veux consulter le tableau de bord global avec le CA de tous mes garages.	CA global et par garage, filtre période, graphiques affichés.	8	Must	S6
US28	En tant qu'Admin, je veux voir le taux d'acceptation des devis par garage.	Taux calculé en %, affiché par garage et globalement.	5	Must	S6

Suite page suivante...

ID	User Story	Critères d'acceptation	SP	Priorité	Sprint
US29	En tant qu'Admin, je veux consulter le rendement de chaque mécanicien.	Nombre interventions, temps moyen, taux de satisfaction affichés.	5	Should	S6
US30	En tant qu'Admin, je veux recevoir un rapport hebdomadaire automatisé par email.	Email envoyé chaque lundi via n8n, contient KPIs clés.	5	Should	S6
EPIC 7 — Infrastructure, DevOps et Sécurité					
US31	En tant que DevOps, je veux conteneuriser tous les microservices avec Docker.	Dockerfile pour chaque service, docker-compose fonctionnel, variables d'environnement externalisées.	8	Must	S7
US32	En tant que DevOps, je veux un pipeline CI/CD complet sur GitHub Actions.	Lint, tests, build, scan sécurité, push registry, déploiement automatisé.	13	Must	S7
US33	En tant que DevOps, je veux déployer l'infrastructure via Ansible en zéro-downtime.	Playbooks Ansible fonctionnels, déploiement sans interruption de service, SSL configuré.	13	Must	S7
US34	En tant qu'Admin, je veux surveiller l'état du système via Prometheus et Grafana.	Métriques collectées, dashboards Grafana actifs, alertes configurées.	8	Must	S7
US35	En tant qu'utilisateur, je veux que mes données soient protégées conformément au RGPD.	Export données, droit effacement implémenté, consentement enregistré.	8	Must	S7

2.5 Planification des Sprints

TABLE 2.5 – Planification générale des sprints

Sprint	Contenu	Durée	Total SP
S1	Authentification, multi-tenant, gestion garages et utilisateurs	2 sem.	26
S2	Gestion clients, véhicules, sélection garage	2 sem.	14
S3	Demandes d'intervention, diagnostic IA, affectation	3 sem.	31
S4	Workflow réparation, suivi temps réel	2 sem.	16
S5	Devis dynamiques, gestion de stock	2 sem.	24
S6	Facturation automatique, reporting global	3 sem.	31
S7	DevOps, CI/CD, Ansible, monitoring, RGPD	3 sem.	50
Total		**17 sem.**	**192 SP**

2.6 Diagramme de Gantt — Planification des Sprints

La figure 2.1 illustre la répartition temporelle des sprints sur la durée totale du projet.

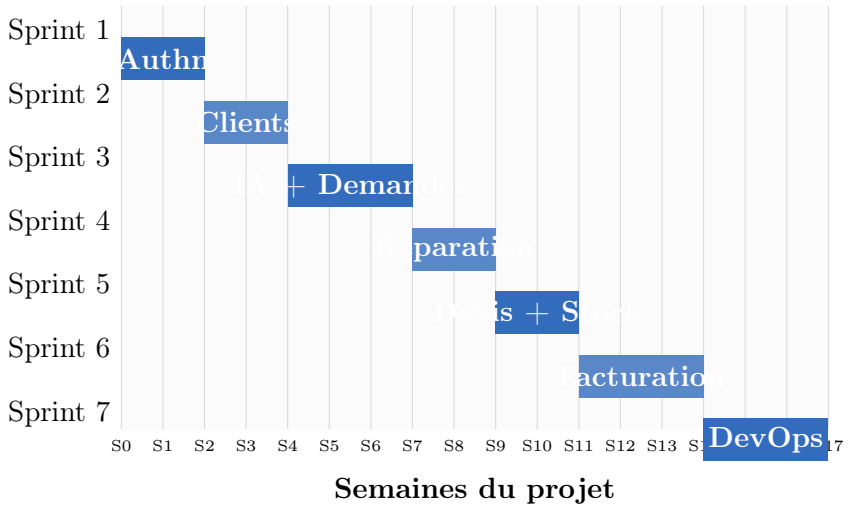


FIGURE 2.1 – Diagramme de Gantt — Planification des sprints MecaManage

2.7 Vélocité et Suivi d'Avancement

La vélocité d'une équipe Scrum correspond au nombre de Story Points réalisés par sprint. Pour MecaManage, la vélocité cible est estimée à **27 SP/sprint** en moyenne,

basée sur la capacité de l'équipe de trois développeurs à temps partiel (projet académique).

Table 2.6 – Vélocité prévisionnelle par sprint

Sprint	SP Prévus	SP Réalisés	Observations
S1	26	—	Mise en place environnement + apprentissage stack
S2	14	—	Sprint court, montante en compétence
S3	31	—	Sprint critique — intégration IA
S4	16	—	Sprint standard
S5	24	—	Complexité métier devis
S6	31	—	Génération PDF + reporting
S7	50	—	Infrastructure critique — effort concentré
Total	**192**	—	

Conclusion du Chapitre 2

Ce chapitre a présenté l'organisation agile du projet MecaManage selon le cadre Scrum. Le backlog produit définit 35 user stories réparties sur 7 sprints couvrant l'ensemble des fonctionnalités du système, pour un total de 192 Story Points. La planification itérative garantit des livraisons incrémentales et une capacité d'adaptation continue aux évolutions des besoins. Le chapitre suivant présentera la conception détaillée de l'architecture du système.